

PUBLIC SAFETY MAP

공공안전지도

흩어져 있던 치안 인프라 정보를 격자 단위로 통합하고,
돌발 상황을 실시간으로 알려주는 시민 참여형 안전 지도



TEAM

3조 SAFETY

MEMBERS

윤영빈 · 남협 · 최정봉 · 박상우 · 최민기

CONTENTS

목차

I.	기존 서비스의 문제점	3
II.	프로젝트 개요	6
III.	활용 데이터	10
IV.	아키텍처	13
V.	기능 소개	19
VI.	트러블슈팅	37
VII.	느낀점	53

01 기존 서비스의 문제점

01

생활안전지도

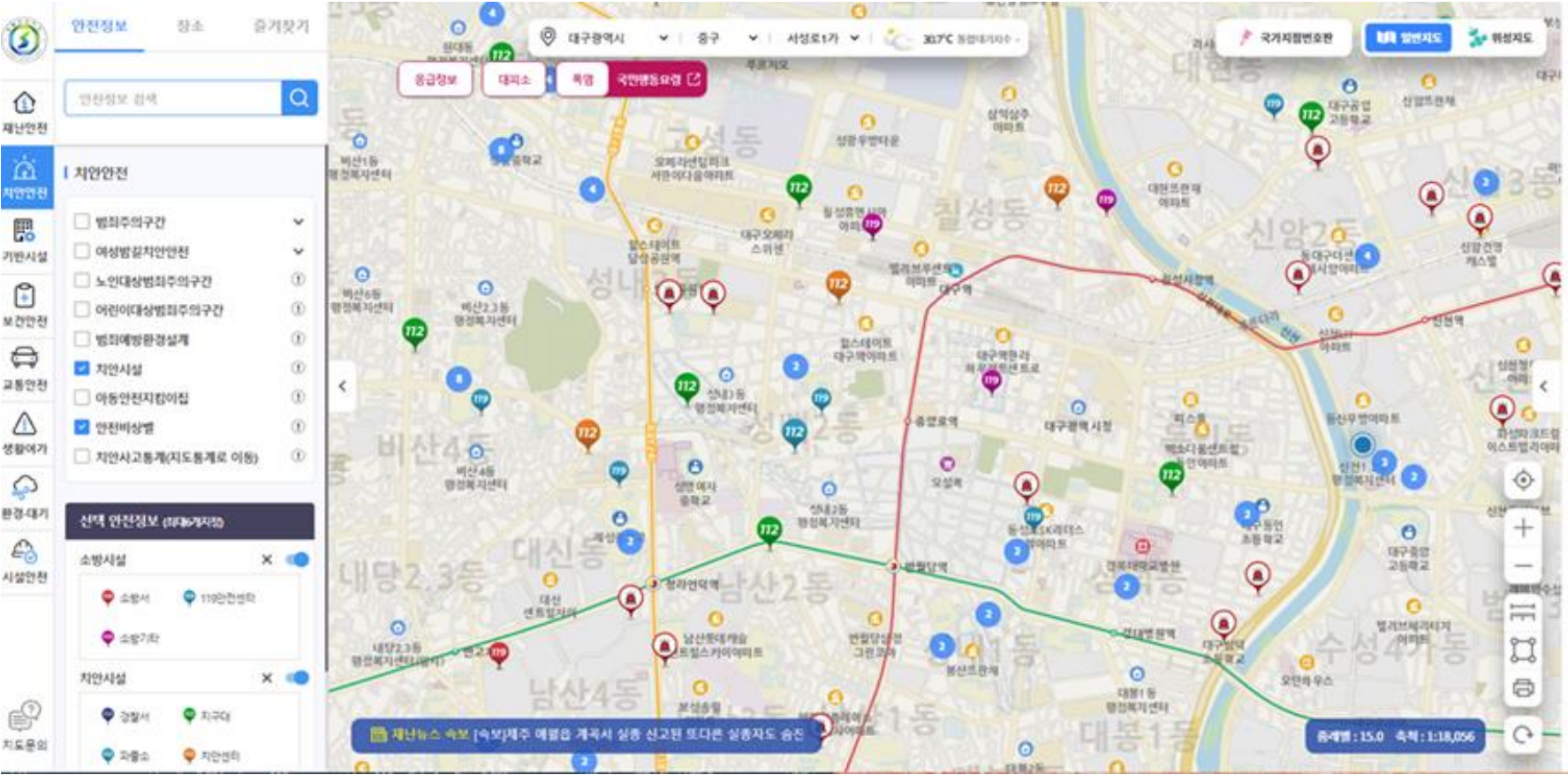
02

기존 서비스 불편사항

기존 서비스의 문제점

생활 안전 지도

전국 생활안전 정보를 하나의 지도에 통합해 제공하는 서비스입니다.



https://www.safemap.go.kr/main/smap_renewal.do

주요 기능

- **범죄주의구간 표시**
범죄 주의 지역을 색상으로 구분해 안내
- **안전비상벨 안내**
위급 상황 시 이용 가능한 비상벨 위치 표시
- **아동안전지킴이집 확인**
어린이 보호를 위한 지킴이집 위치 확인

기존 서비스의 문제점

생활 안전 지도의 불편사항

#01 정보 분산

치안 정보가 여러 곳에 흩어져 있다

CCTV, 보안등, 파출소 등 치안 인프라 정보가
여러 출처와 페이지에 분산되어 있어 한 번에 확인이 어렵습
니다.

#02 직관성 부족

해당 구역이 안전한지 한눈에 파악하기 어렵다

개별 인프라 위치는 보이지만, 지역의 종합 안전도가
직관적으로 시각화되어 있지 않습니다.

#03 실시간성

돌발 상황이 즉시 반영되지 않는다

싱크홀, 공사, 사고 등 돌발 상황이
기존 지도 서비스에 실시간으로 반영되지 않습니다.

#04 지표 괴리

정량 수치와 실제 체감 안전도가 다를 수 있다

인프라 개수 같은 정량 지표만으로는 실제 거주자가 느끼는 안전도를 담기 어렵습니다.
같은 CCTV 개수라도 시간대·골목 구조에 따라 체감은 완전히 달라집니다.

#05 사용자 채널 없음

체감 안전도를 확인할 창구가 없다

실제 그 지역에 사는 사람이 느끼는 안전/불안 신호를
다른 사람에게 전달할 수 있는 공식 창구가 존재하지 않습니다.

02 프로젝트 개요

01
제작의의

02
제공 서비스

03
App 핵심 기능

프로젝트 개요

제작의의

PROBLEM

① 정보가 흩어져 있음

CCTV·보안등·소방시설·파출소·편의점 등 치안 정보가 여러 페이지에 분산

② 실시간성이 없음

사고·공사·싱크홀 등 돌발 상황이 지도에 늦게 반영

③ 체감 안전도 확인 불가

실제 거주자의 안전 감각을 담은 채널이 없음

SOLUTION

공공데이터를 격자 단위로 통합

CCTV·보안등·소방시설·파출소·편의점을 0.01° 격자로 집계

사고다발구역 + 유저 제보 결합

KOROAD 사고다발구역 접근 감지 + 사용자의 실시간 돌발상황 제보

격자별 체감안전도 평가

불안 / 보통 / 안전 3단계 · 태그와 한줄평 등록

USER EXPERIENCE

지도 위 안전등급 컬러 시각화

한 번 지도에 진입하면 그 동네가 안전/보통/불안 중 어디에 속는지 즉시 인식

400m반경 내 사용자에게 푸시

Firebase Cloud Messaging + 앱 측 거리 계산으로 위치 기반 알림

거주자 목소리가 지도에 쌓임

정량 데이터 +정성 피드백을 함께 반영해 지역 안전도를 보완

프로젝트 개요

제공하는 서비스

- 지원
- 미지원

#	기능	설명	Web	App
01	안전등급 지도	격자별 안전/보통/불안 컬러 시각화	●	●
02	인프라 마커	CCTV · 보안등 · 파출소 · 소방시설 · 편의점	●	●
03	사고다발구역	KOROAD 4종 위험구간 표시	●	●
04	제보	사고 · 공사 · 싱크홀 등 실시간 제보 작성	○	●
05	체감안전도 피드백	격자별 태그 · 한줄평 작성	○	●
06	계정 · 마이페이지	인증 · 내 활동 관리	●	●
07	관리자 콘솔	제보 · 피드백 · 도시정보 · 데이터 관리	●	○
08	알림 · 길안내 · 오프라인	근접 알림 · TMAP 경로 · SQLite 로컬	○	●

프로젝트 개요

APP 핵심 기능

01 TAG

격자별 피드백 태그 작성

지도 위 격자를 선택 → 태그와 한줄평 작성 → 그 지역의 체감 안전도 데이터가 지도에 반영됩니다.

02 REPORT

돌발 상황 제보 작성

현재 위치 기반 · 유형별 아이콘 · 사진 첨부. 이 데이터가 반경 400m 사용자에게 즉시 전달됩니다.

03 NAVIGATION

3가지 안전 경로 탐색

A

CCTV 경유

B

최단거리

C

제보 최소화

최단거리 대비 소요시간이 **1.3배**를 넘지 않는 경로만 후보로 제시

04 PROXIMITY ALERT

근접 알림

400m

알림 반경

- 사고다발구역 **400m 이내** 접근
- 다른 유저의 제보 **400m 이내** 접근

03 활용 데이터

01

데이터 수집 · 스코어링

02

안전 등급 모델

활용 데이터

데이터 수집부터 스코어링까지 4단계

운영에는 3단계 규칙 기반 공식만 Backend에 이식되어 매일 자정 재계산됩니다. 4단계 ML은 참고용.

1STEP

데이터 수집

CCTV	≈ 370,000
경찰시설	≈ 3,000
소방시설	≈ 2,000
편의점	≈ 54,000
KOROAD 사고다발	4중

2STEP

전처리

- 인코딩 자동 판별
- 좌표계 정규화
- 국내 범위 클리핑
- 시설 유형별 가중치
- 0.01° 격자 집계

3OPERATIONAL

베이스라인 스코어링

- 시설군 정규화

→ infra_score

→ safety_baseline

→ event_penalty

→ safety_strength

DEPLOYED TO

Backend · 매일 자정 cron

4REFERENCE ONLY

참고용 ML

- RandomForestRegressor
- MAE · R²
- Feature Importance

NOT IN PRODUCTION

운영 서버에는 연결하지 않고 참고용 산출물
로만 사용

활용 데이터

안전 등급 모델

·	데이터 수집 infrastructures + report — DB 직접 조회, CSV 미사용
·	격자별 집계 grid_id 기준 시설 수 · 활성 제보 · 최근 180일 이력 제보 카운트
·	인프라 점수화 log1p(시설 수) + 희소시설 보너스 → 시설군별 z-score 정규화
·	가중합 · 백분위 CCTV 11% · 경찰서 38% · 소방서 35% · 편의점 16% 가중합 → percentile
·	감점 반영 활성 제보 $8 \times \log_{1p}(N)$ + 이력 제보 최대 5점($1.5 \times \log_{1p}$) 차감
·	등급 산출 safety_strength 상대 3분위(불안·보통·안전) → 매일 00시 배치 UPDATE

SAFETY STRENGTH — WEIGHTS

시설군 가중치

CCTV 11% · 경찰서 38% · 소방서 35% · 편의점 16%

log1p(시설 수)로 압축 후 시설군별 z-score 정규화 → 가중합. 경찰서·소방서는 존재 시 +0.5 보너스

SAFETY STRENGTH — GRADE

등급 산정 — 상대 3분위

● 하위 1/3 불안 ● 중위 1/3 보통 ● 상위 1/3 안전

안전강도 = 인프라 점수 – 제보 감점 – 이력 감점

매일 00:00(Asia/Seoul) 배치로 재계산 · CLI 수동 실행 / --dry-run 지원

04

아키텍처 및 개발 과정

01

시스템 아키텍처

02

Database ERD

03

Backend 기술 스택

04

개발 프로세스

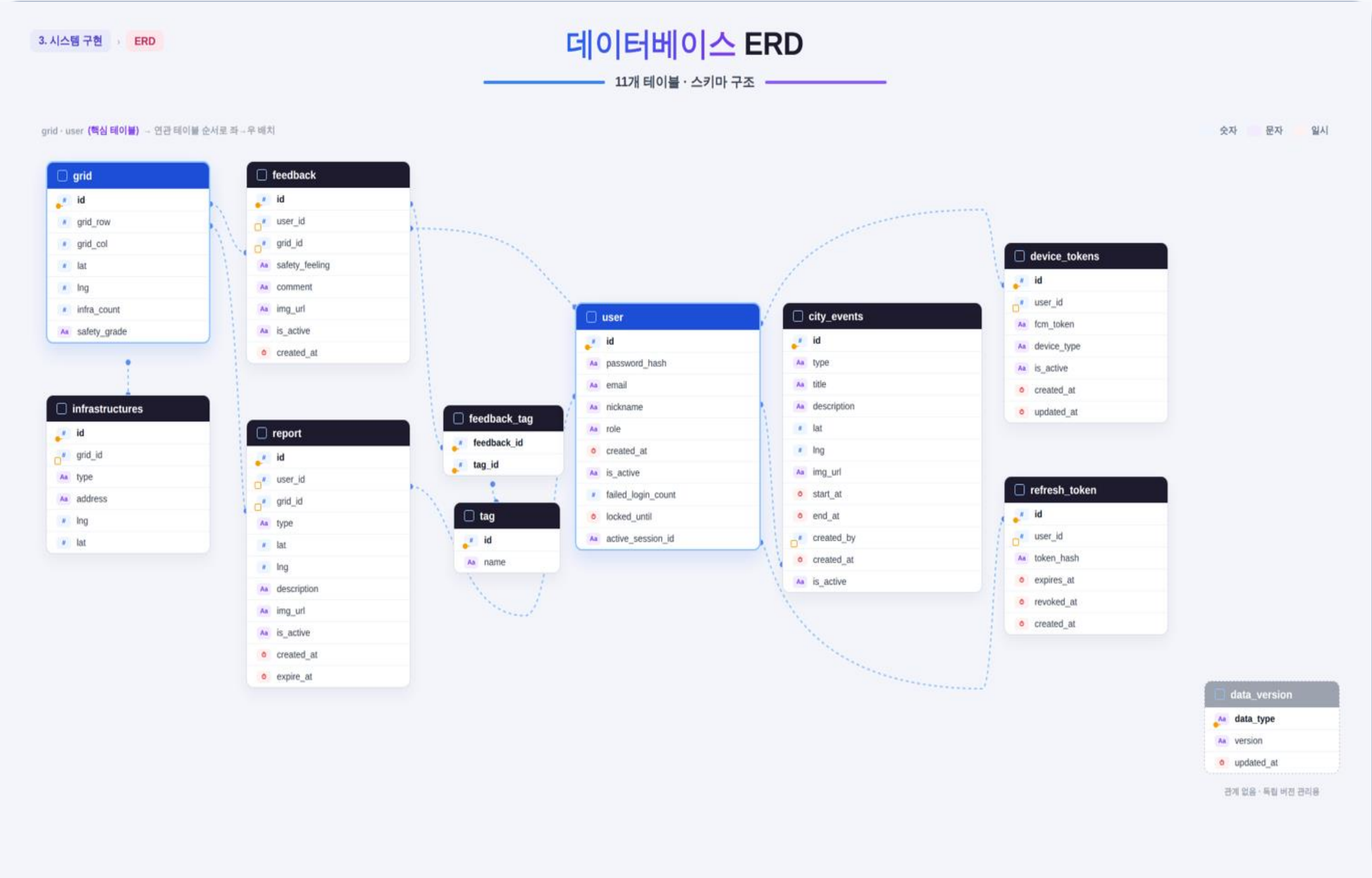
아키텍처 및 개발 과정

외부 API 4종과 Web / App / Backend



아키텍처 및 개발 과정

DB 구성 테이블 | ERD



DOMAIN MAP

CORE **grid**

infrastructures · feedback · report 를 잇는 격자 마스터
(안전 등급의 최소 단위)

USER **user**

city_events · device_tokens · refresh_token 과 연관
(인증·알림·사용자 활동 근원)

FEEDBACK **feedback**

feedback_tag ↔ tag 를 통해 다대다 태그 관계 구성

STANDALONE **data_version**

앱 SQLite 로컬 데이터 버전 관리용 (독립 테이블)

아키텍처 및 개발 과정

REST 백엔드 · AI 백엔드 역할별 분리

REST BACKEND

Node.js 기반 REST API 서버

Web · App 클라이언트가 사용하는 공용 API 서버

Node.js

TypeScript

Express.js

Prisma ORM

JWT +Session

bcrypt

RESPONSIBILITIES

- 사용자·관리자 인증
- 격자 · 인프라 · 제보 · 피드백 CRUD
- FCM 발송 · 신규 제보 브로드캐스트

AI BACKEND

Python 데이터·모델 파이프라인

전처리 · 스코어링 · 참고용 모델 학습을 담당

Python

pandas

NumPy

scikit-learn

joblib

OpenCV

YuNet (ONNX)

RESPONSIBILITIES

- 공공데이터 전처리 · 격자 집계
- safety_strength 산출
- 참고용 ML 모델(RandomForest) 학습·평가
- openApi YuNet을 이용한 사진 모자이크 처리

아키텍처 및 개발 과정

개발 착수 5단계

1

STEP 1

기능 정의

누가·왜·무엇을 쓸지 사용자 시나리오부터 정리

2

STEP 2

UI 설계

Web·App 화면 흐름과 인터랙션 시나리오를 스케치

3

STEP 3

DB 테이블 설계

grid·user·feedback 등 11개 테이블 관계를 확정

4

STEP 4

API 명세서

Backend REST API 엔드포인트와 스키마를 문서화

5

STEP 5

개발 시작

앞선 4단계 산출물을 기반으로 실제 코드 작성 착수

FRONTEND STACK

Next.js 앱 라우터 기반 웹 +Flutter 앱

Next.js App Router

React 19

Zustand

Kakao Maps SDK

Firebase

아키텍처 및 개발 과정

지도 → 격자 → 인프라 → 안전등급 → 사고다발구역 → Flutter

STEP 01

지도 기본

Kakao Maps · mapStore · 주소
소 검색

STEP 02

격자 · 마커 UX

Polygon · 패널 · 인프라

STEP 03

사고다발구역

KOROAD API 4종 연동

STEP 04

안전등급 산출

safety_strength · 매일 자정 cron
cron

STEP 05

Flutter 이식

웹 UI · GPS · 클러스터링

STEP 06

제보 · 네비

회피 경로 · 재안내

STEP 07

알림

근처 제보 · 도착 알림

SCOPE HIGHLIGHTS

Web + App

웹과 앱 양쪽의 지도·격자·인프라 UI 담당

daily cron

매일 자정 안전등급을 재계산하는 배치를 구현

KOROAD

보행자·자전거·이륜차·어린이보호구역 4종 위험구간 연동.

05

주요 기능

5-1

공통

지도 · 로그인 · 알림

5-2

APP

제보 · 피드백 · 알림 · 마이페이지 안내 UX

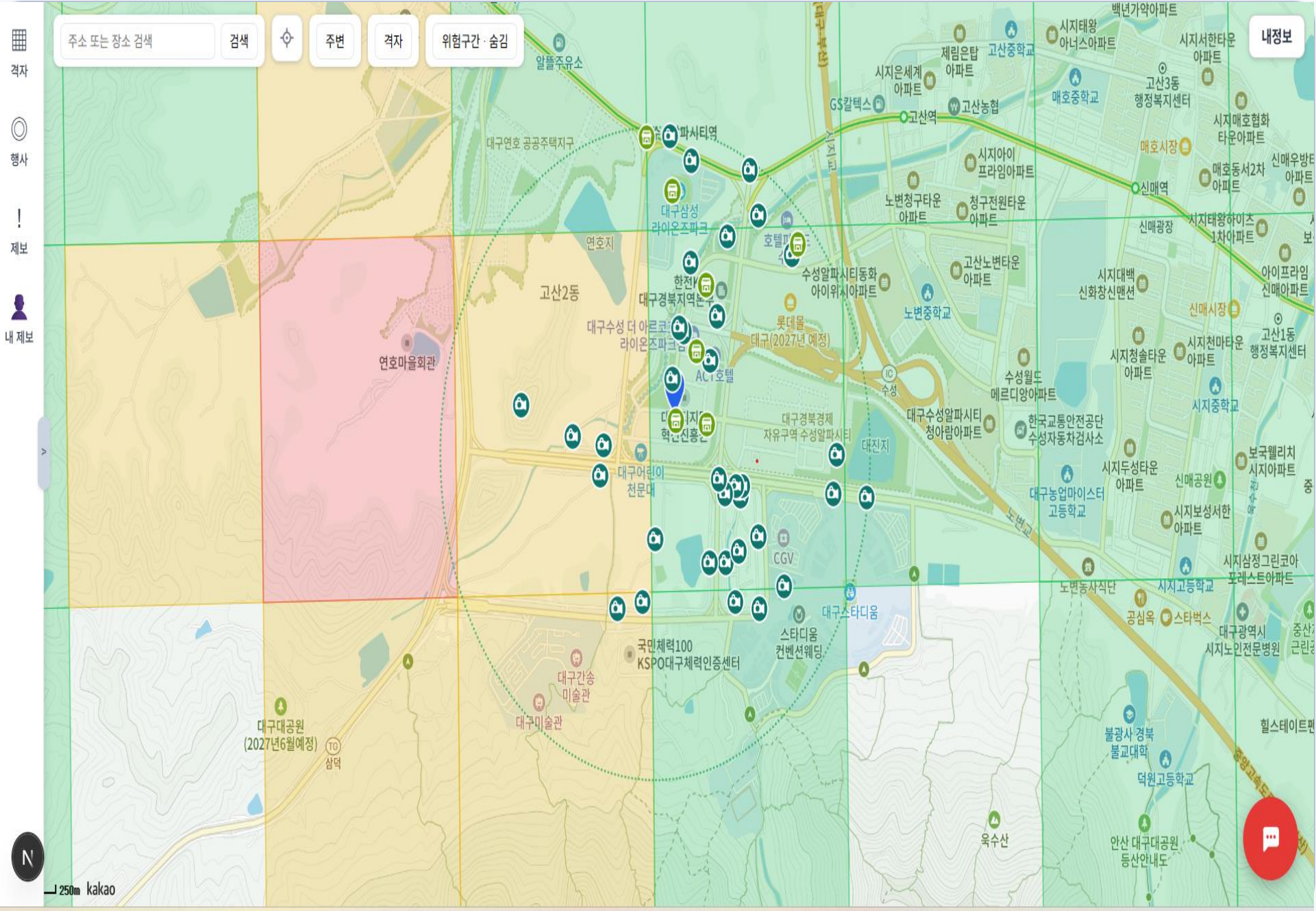
5-3

WEB

내 제보 / 피드백 · 관리자 페이지 · 채팅

주요 기능 : 인프라, 안전등급 지도 표시

지도에 안전 인프라, 등급 표시



IMPLEMENTATION

- Kakao Maps 연동**
 - 지도 렌더링 · 이벤트 바인딩 · 컨트롤 UI
- mapStore (Zustand) 상태 관리**
 - 뷰포트·격자·마커 상태를 전역에서 참조
- /g rids Polygon 렌더링**
 - 뷰포트 안 격자만 그려서 렌더링 비용 절감
- /infrastructures 인프라 마커**
 - CCTV · 보안등 · 파출소 · 소방 · 편의점 마커
- 과도한 축소 시 격자 대량 생성 방지**
 - 축소 레벨 상한 · 뷰포트 제한 로직
- 주소 검색 → 지도 이동**
 - 지오코딩 후 대상 좌표로 카메라 이동
- 내 위치 이동**
 - 브라우저 Geolocation → 카메라 재조준

주요 기능 : 인프라, 안전등급 지도 표시

뷰포트 기준 마커와 3종의 상세 패널

3 TYPES OF DETAIL PANEL

GRID

격자 상세 패널

선택한 격자의 인프라 요약·안전등급·태그 통계를 한 패널에서 확인

EVENT

행사 상세 패널

지역 축제·공사 등 행사 이벤트 위치와 기간, 상세 정보

REPORT

제보 상세 패널

사용자 제보의 유형·시각·설명을 확인하는 상세 패널

MARKER LOGIC

뷰포트 기준 행사·제보 마커

현재 지도 화면에 들어오는 행사·제보만 그려서 마커 폭주를 방지합니다.

viewport < map.getBounds() → 마커 필터링
→ 화면 밖 마커는 렌더링 스킵

TOGGLE

인프라 타입별 ON/OFF · 피드백 태그

- 인프라 타입별로 마커 표시 여부 즉시 토글
- 격자 상세 패널에서 체감안전도 피드백 태그 확인

주요 기능 : 인프라, 안전등급 지도 표시

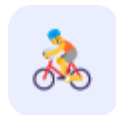
KOROAD 4종 위험구간 지도 시각화

DATASETS · 4



보행자

보행자 사고다발지점



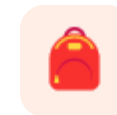
자전거

자전거 사고다발지점



이륜차

이륜차 사고다발지점



어린이보호구역

스쿨존 사고다발지점

API CALL OPTIMIZATION

호출량을 줄이기 위한 2가지 조건

CONDITION 01

구(區) 변경 감지

같은 구 안에서는 반복 호출하지 않고 캐시. 구가 바뀔 때만 신규 호출로 API 사용량을 크게 절감.

CONDITION 02

일정 줌 이상 호출/표시 중단

전국 단위 축소 시에는 사고 마커가 시각적 의미를 잃으므로 API 호출과 렌더링을 모두 중단.

주요 기능 : 인프라, 안전등급 지도 표시

매일 자정, 격자별 등급을 다시 계산

CALCULATION FLOW

격자별 점수 → 감점 → 최종 등급



SCHEDULE

매일 자정 cron

0 0 * * * → npm run safety:recalc

MANUAL

수동 CLI 실행 지원

운영 중 필요 시 CLI로 즉시 재계산.
배치 로직은 node-cron 스케줄러와 동일한 함수 사용.

FORMULA

$\text{safety_strength} = \text{infra_score} - \text{event_penalty}$

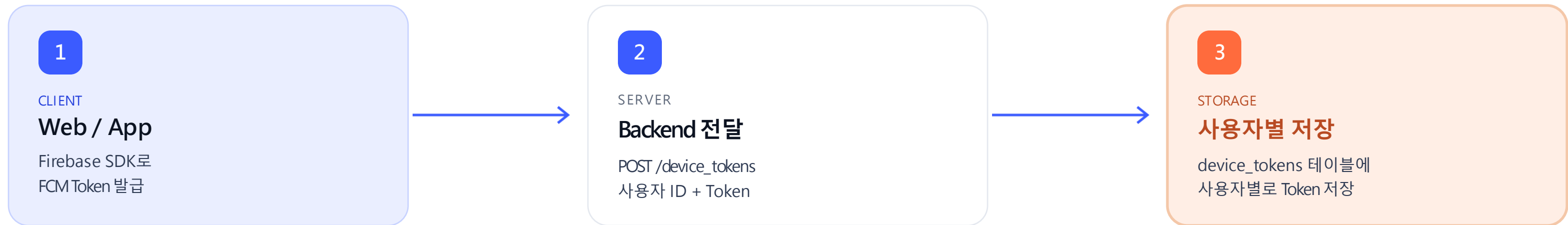
동일한 공식이 웹·앱 지도의 격자 색상을 결정합니다.

주요 기능 : 알림

Firestore를 활용한 알림 서비스

3-STEP FLOW

Client → Backend → Database



WHY THIS DESIGN

한 사용자가 여러 기기에서 접속해도 각 기기의 토큰을 각각 관리

- 사용자 ID 기준으로 여러 토큰을 저장하여 알림이 사용자의 모든 기기에 도달합니다.
- 토큰이 만료되면 다음 접속 시 자동 갱신되어 항상 유효한 토큰만 유지.

주요 기능 : 알림

전송은 전체에게, 표시는 400m 이내에서만

ADMIN · WEB

관리자만 수신

신규 제보 발생



Backend



FCM → 관리자 role 대상



관리자 알림 표시

관리자로 로그인한 계정에만 도달하도록 서버 측에서 필터링

USER · APP

전체 전송 · 앱에서 거리 필터

신규 제보 발생



Backend → FCM → 전체 사용자



App: 현재 위치와 제보 위치 거리 계산



400m ≤
YES → 알림 표시

400m >
NO → 미표시

IMPORTANT

FCM 메시지는 전체 사용자에게 전송하고, 실제 알림 표시 여부는 App에서 거리 계산으로 결정합니다.

서버는 누구에게 전달할지"를 몰라도 되고, 위치 프라이버시는 클라이언트에서만 처리됩니다.

주요 기능 : 로그인 인증

관리자와 일반 사용자에게 다른 인증 방식을 적용

AADMIN

Session 기반 인증

Session ID 저장 위치Database

중복 로그인기존 세션 만료

특징서버 측 무효화 가능

동일 계정 중복 로그인 시 기존 세션을 강제로 만료시켜 관리자 계정 탈취 위험을 최소화.

UUSER

JWT 기반 인증

Access Token30분

Refresh Token7일

만료 시 처리Refresh 검증 후 재발급

Access 만료 시 자동 재발급 로직으로 로그인 유지. 세션 저장 없이 확장성 확보.

MIDDLEWARE FLOW

모든 요청은 미들웨어에서 인증을 확인한 뒤 백엔드로 전달됩니다

01
사용자 요청

→

02
Middleware

→

03
인증 확인

→

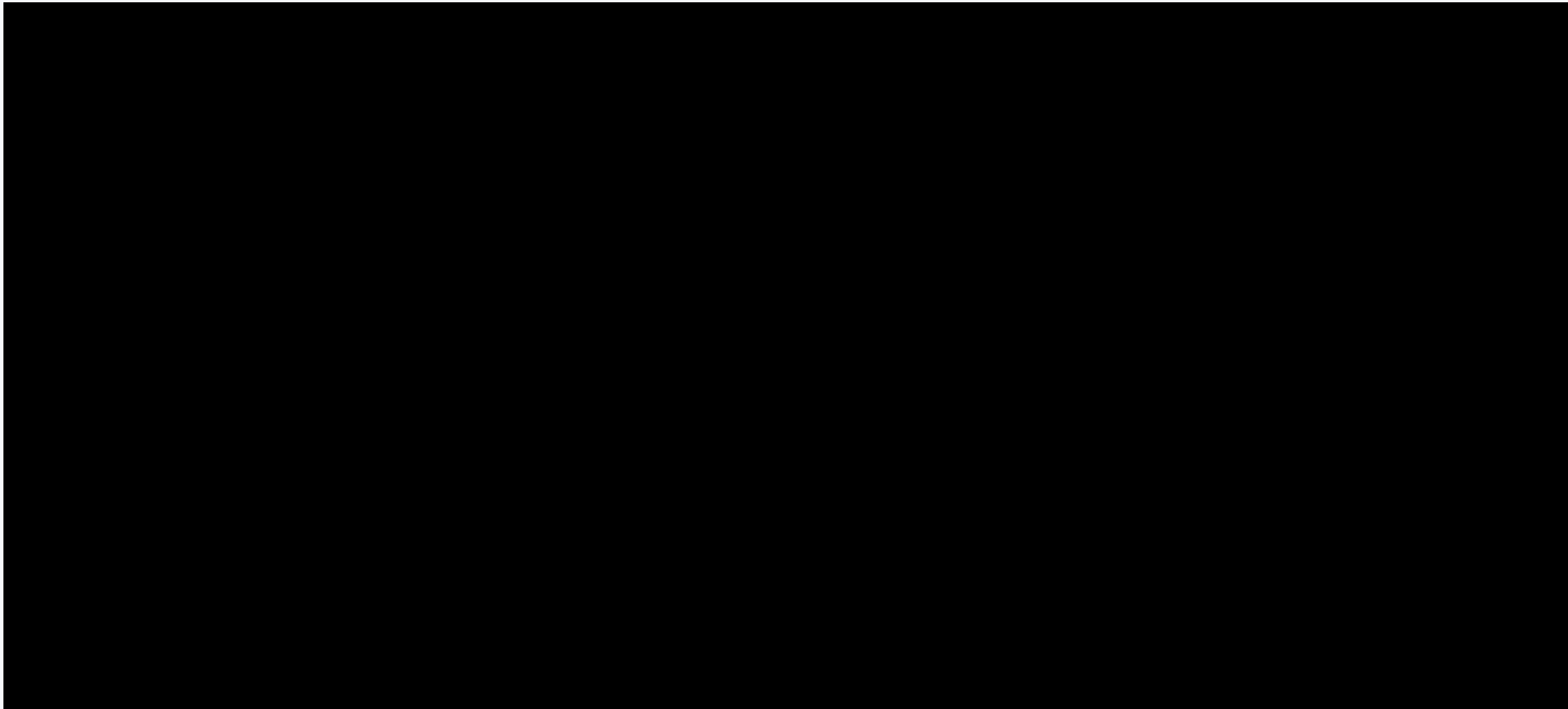
04
Backend

→

05
Database

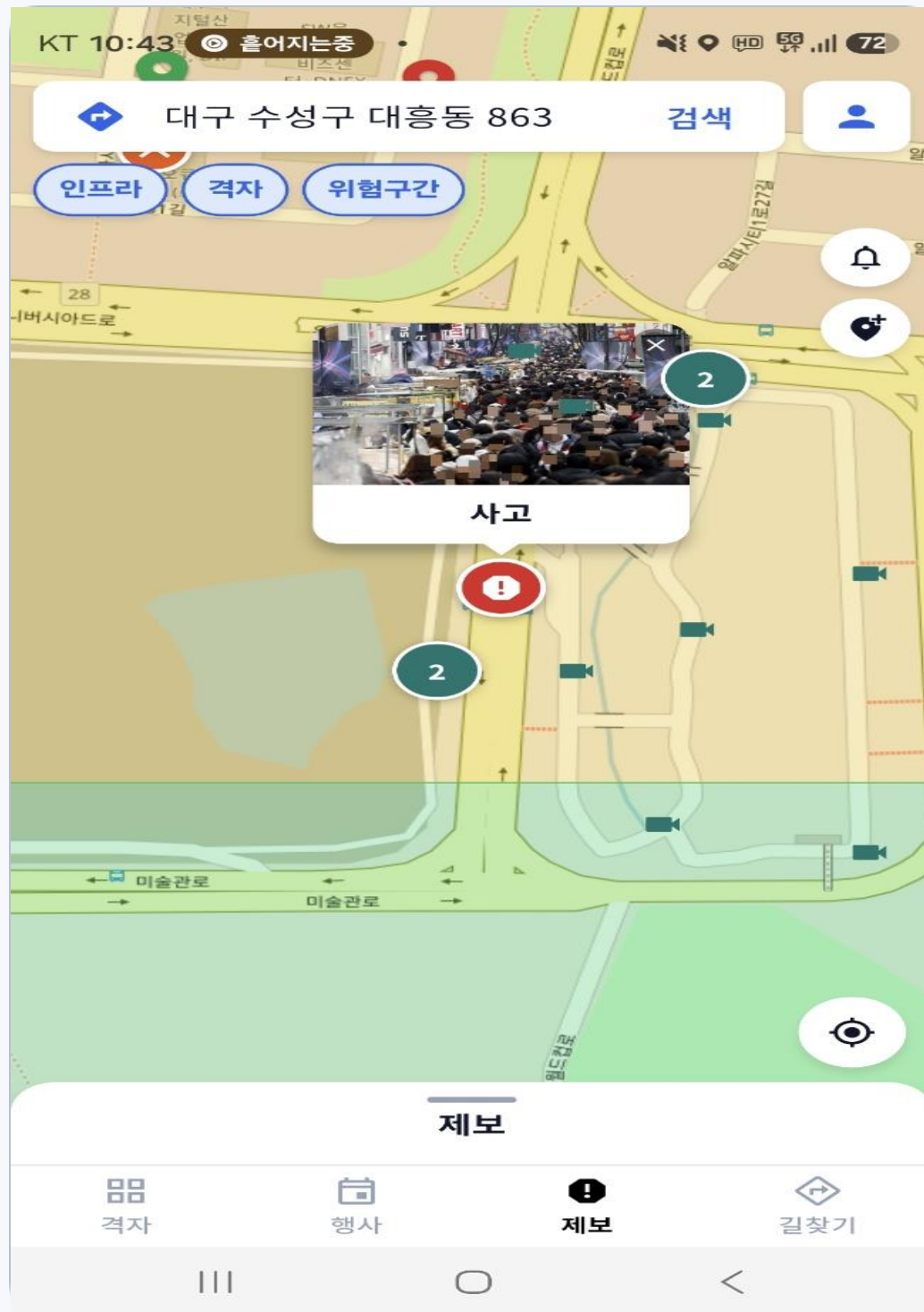
주요 기능 : 로그인 인증

관리자 로그인 - 다른 곳에서 동일 아이디로 로그인 발생 시



주요기능 : APP

웹의 지도 경험 + 앱만의 기능



FEATURES

MIRROR

웹 지도 구조 및 UI 이식

격자 · 인프라 마커 · 패널 UX를 앱에서 동일한 형태로 재현

REPORT

제보 유형별 마커 아이콘

사고 · 공사 · 싱크홀 등 유형별 아이콘으로 즉시 인지

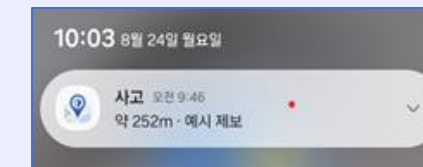
GPS

실시간 GPS · 카메라 추적

현재 위치 추적 및 카메라 자동 이동

ALERT

근처 제보 400m 알림



CLUSTER

CCTV 클러스터링

약 37만 개 마커를 줌 레벨별로 묶어서 성능 확보

ROUTE

위험구간·제보 회피 경로 +재안내

경로 이탈 시 자동 재안내, 목적지 접근 알림

주요기능 : APP

3가지 길안내 UX

1

FUNCTION 01

롱프레스 목적지 지정

지도의 어느 지점이든 길게 눌러 목적지로 지정합니다. 주소는 검색창에 자동 반영되고, 복사도 가능.

2

FUNCTION 02

경로 탐색 · 미리보기

출발지 · 도착지를 확정하면 경로 후보 3종을 지도 위에 미리보기로 표시. 확인 후 안내 시작.

3

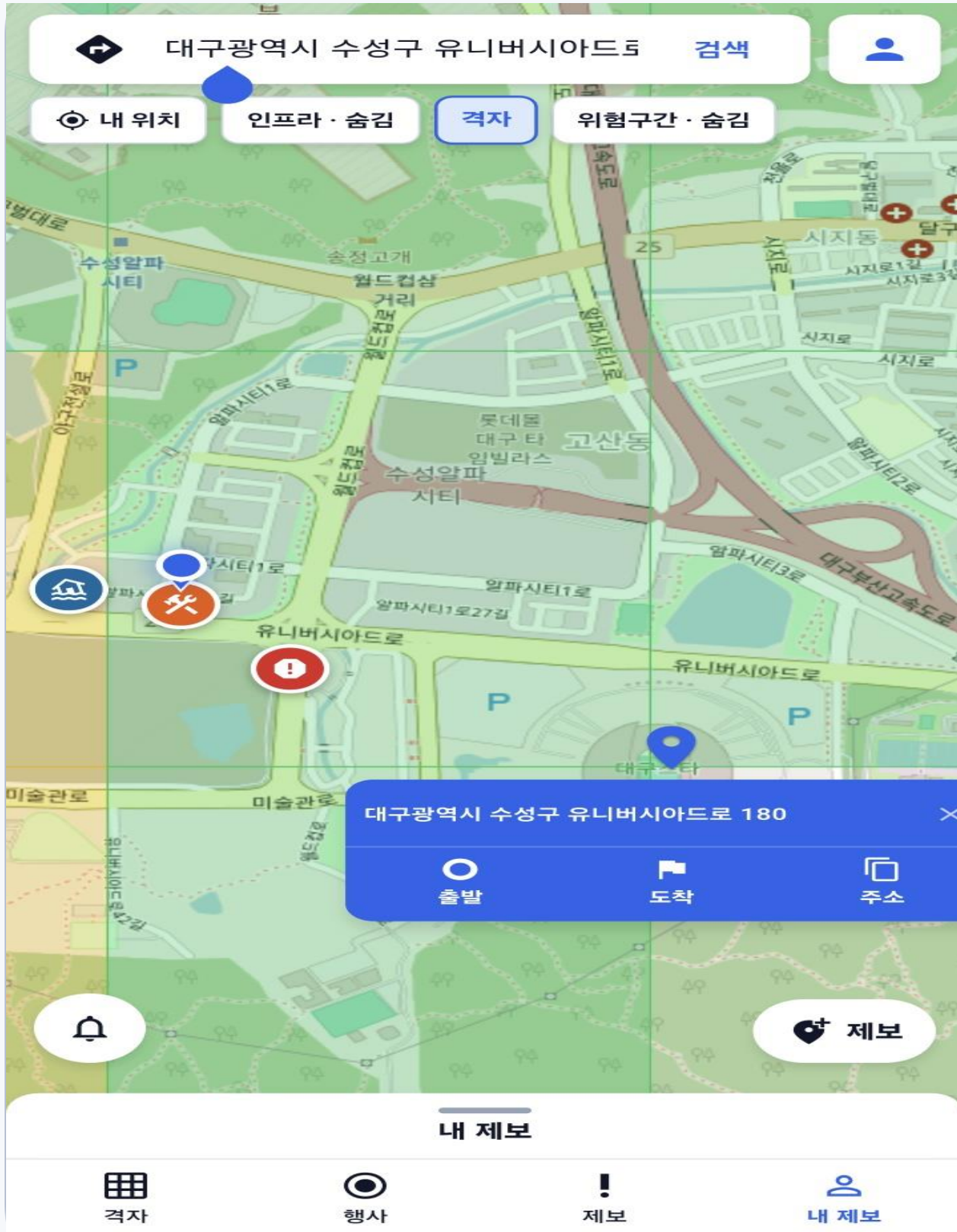
FUNCTION 03

안내 시작 · 종료 UX

안내 중에는 격자를 자동 OFF, 새 목적지 재지정, 안내 종료 시 패널이 다시 뜨지 않도록 세밀하게 제어.

주요기능 : APP

롱프레스 기능 도입



STEP BY STEP

- 지도 롱프레스**
임의 좌표에서 롱프레스 제스처 감지

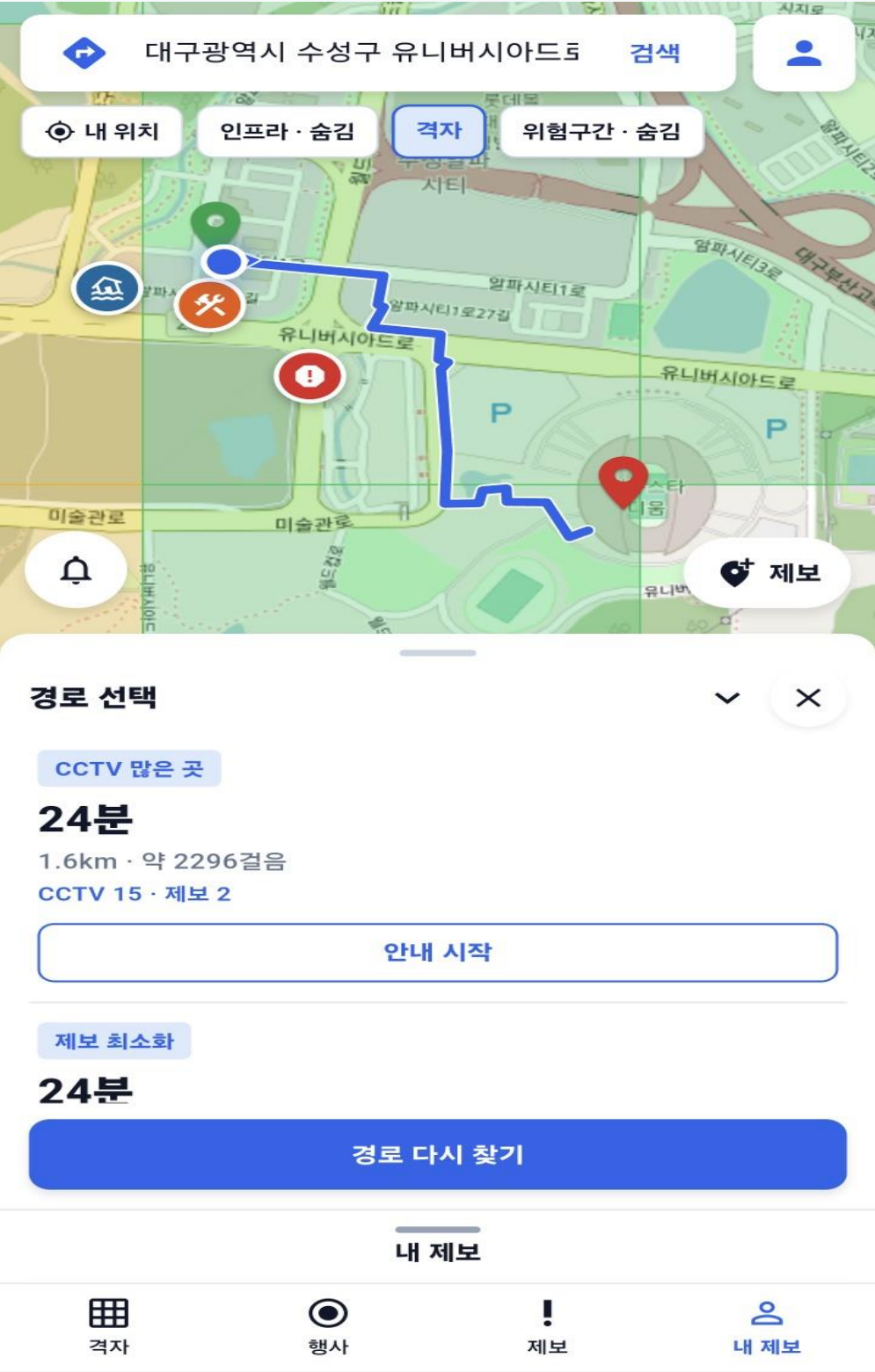
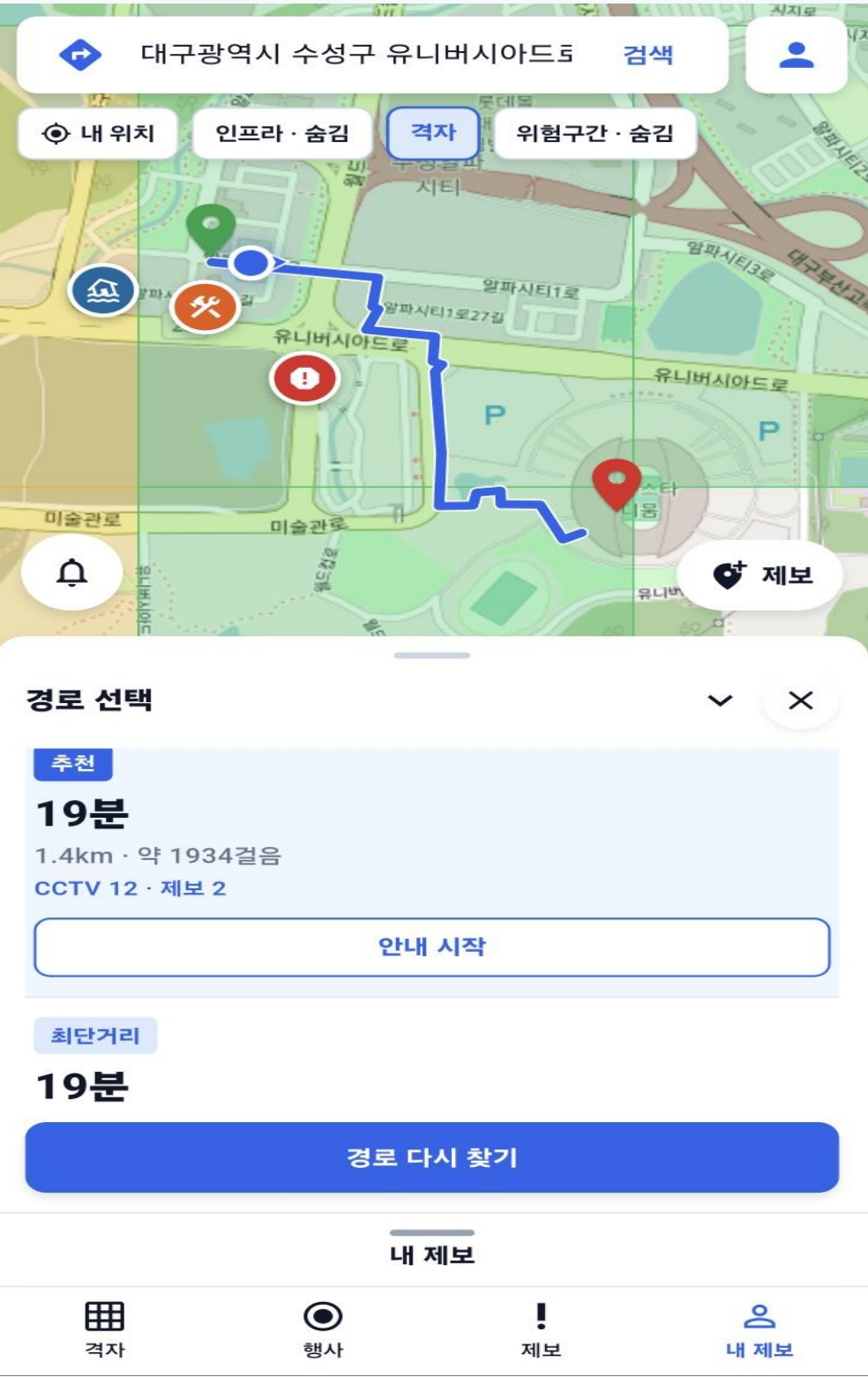
- 선택 UI 노출**
이 위치로 길찾기 · 주소 복사 등 액션 카드 표시

- 주소 검색창 자동 반영**
지오코딩된 주소가 검색창에 즉시 채워짐

- 주소 복사 지원**
공유·메모 등으로 바로 활용 가능

주요기능 : APP

경로 후보 선정



USER FLOW

- 1

출발지 · 도착지 입력

주소 검색 또는 현 위치, 롱프레스 결과 활용
- 2

경로 선택 패널

CCTV 경유 / 최단 / 제보 최소화 3가지 후보 제공
- 3

지도 위 미리보기

선택한 경로를 지도에 실제로 그려서 사전 확인

주요기능: APP

네비게이션 기능

RULE 01 START

안내 시작 시 격자 자동 OFF

경로가 격자에 가려지지 않도록 안내 시작과 동시에 격자 표시를 자동 해제합니다.

RULE 02 RE-ROUTE

새 목적지 재지정 지원

안내 중이라도 지도 롱프레스로 새 목적지를 지정하면 기존 안내를 정상 종료 후 새로 시작합니다.

RULE 03 END

기존 안내 정상 종료

경로 세션·리스너·상태를 명확히 정리한 뒤 종료. 다음 경로가 잔여 상태에 오염되지 않도록 보장.

RULE 04 PANEL

종료 후 패널 재노출 방지

안내 종료 시 경로 선택 패널이 다시 뜨지 않도록 상태를 명시적으로 초기화.
(관련 트러블슈팅은 섹션 6에서 상세히 다룹니다.)

주요기능 : A P P

내 제보·피드백·알림을 한 화면에서 관리합니다

01 REPORT

제보 관리

내 제보 확인 · 수정 / 삭제 · 내 제보 → 해당 위치 확인

02 FEEDBACK

피드백 관리

내 피드백 확인 · 수정 / 삭제 · 위치 확인

03 NOTIFICATION

알림 관리

받은 알림 목록 · ON / OFF 설정

주요기능: WEB

제보 · 도시정보 · 데이터를 한 곳에서 운영합니다

MODULE 01 REPORT · FEEDBACK

제보 · 피드백 운영

날짜별 확인 · 표시/숨김 · 신규 제보 확인

MODULE 02 CITY EVENTS

도시정보 관리

등록 · 수정 · 지도 표시/숨김

MODULE 03 DATA · SOFT DELETE

데이터 관리 (Soft Delete)

- 삭제 요청 → **is_active = N** → 서비스 화면에서 숨김
- 실제 데이터 삭제는 DB Scheduler가 별도 처리

주요기능 : WEB

지도에서 실시간 제보

지도 위 메시지 아이콘

간편하게 버튼 클릭 한번으로 채팅방 진입

본인 확인

로그인된 사용자만 채팅방 진입 가능

관리자와 실시간 대화

Supabase Realtime 채널로 지연 없이 양방향 송수신

채팅 3단계

채팅 진입 및 세션 확인

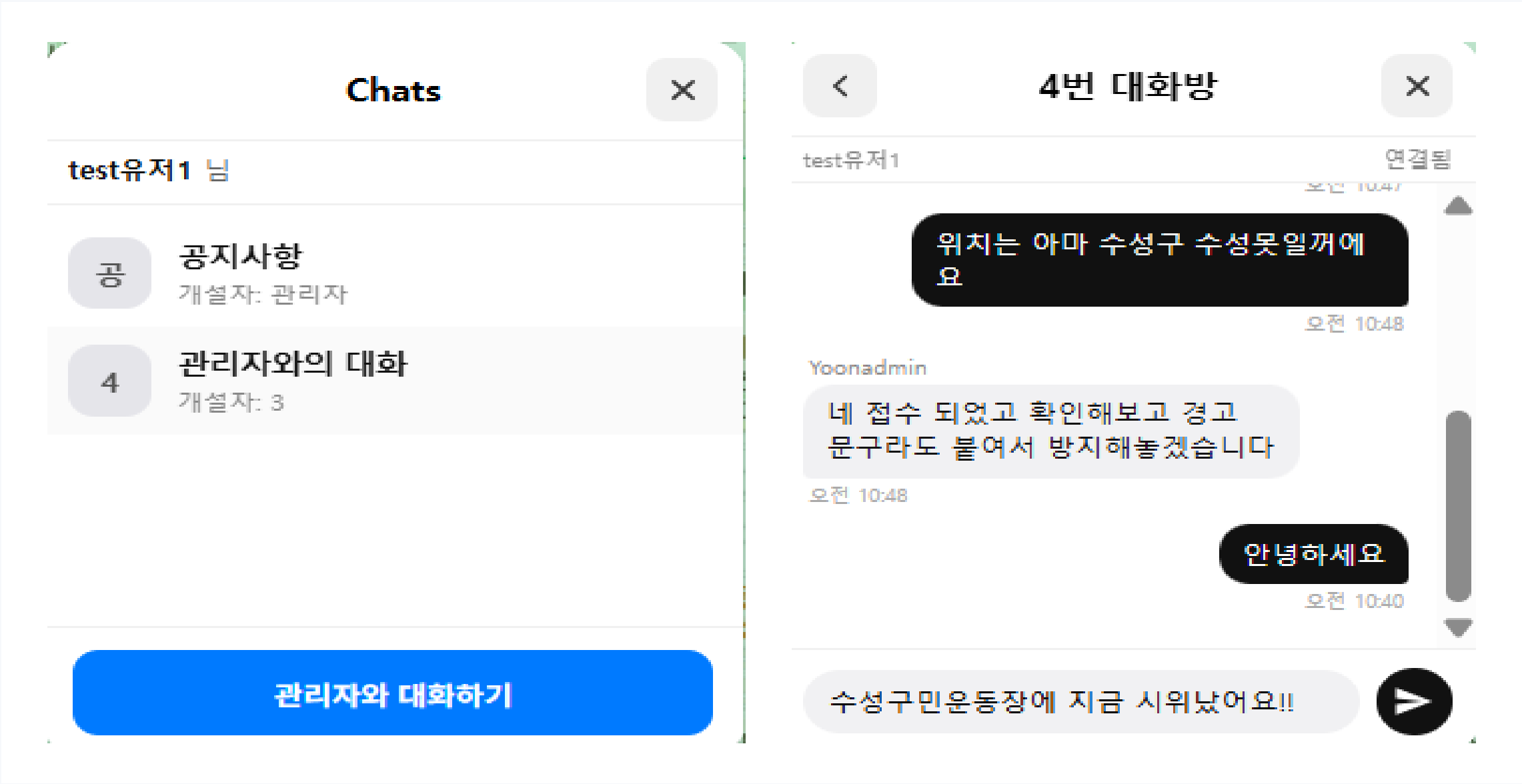
사용자가 채팅에 진입할 때 인증 세션을 검증합니다. 유효 세션이 없으면 로그인으로 유도.

메시지 이벤트 감지

방별 채널을 구독. INSERT·UPDATE 이벤트만 필터링하여 필요한 신호만 수신합니다.

데이터 매핑 및 UI 동기화

sender_id로 닉네임 비동기 매핑 → 상태 업데이트 → 화면 렌더. 배지 카운트도 여기서 갱신.



주요기능 : WEB

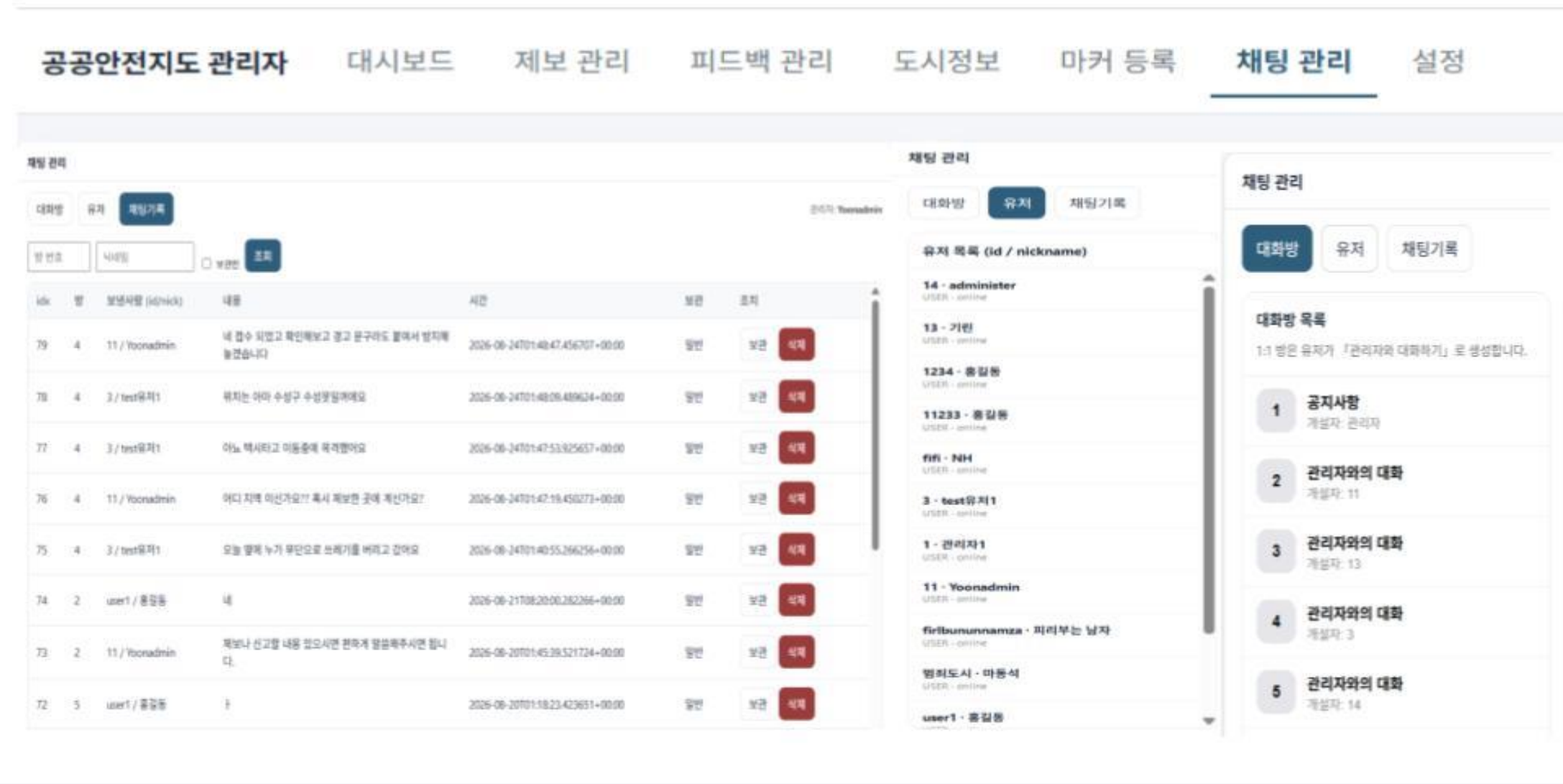
채팅의 주요 기능 및 관리

사용자 대화방 목록

모든 사용자의 대화방을 시간순 정렬로 노출. 미확인 메시지는 상단에 우선 표시하여 응대 누락을 방지합니다.

실시간 메시지 모니터링

선택한 방의 메시지를 실시간 스트림으로 렌더링. Supabase Realtime 이벤트를 필터링해 방별로 분기하여 지연 없이 모니터링할 수 있습니다.



전역 공지 시스템

관리자 작성 → 전 사용자 열람

A 관리자

관제 패널에서 공지 작성 · 발행

U 전 사용자

Web·App 접속 시 즉시 열람 가능

USE CASE

서비스 점검 · 데이터 갱신 · 캠페인 등을 실시간 전파

상태 인지 표시

NEW

새 메시지·새 공지가 도착하면 아이콘에 NEW 표시가 즉시 표시됩니다.

AUTO REFRESH

Supabase Realtime 구독으로 자동 갱신 (수동 새로고침 불필요)

U

ROLE

일반 사용자

관리자와 1:1 Chatting

본인 방만 가능

공지 사항

읽기 전용

타 사용자 대화방

접근 불가

공지 사항 방에서 글 작성

불가

A

ROLE

관리자

전체 대화방 열람

가능

1:1 응답

가능

공지 작성

가능

관제 패널 접근

전용 진입점

06

PROBLEM → CAUSE → FIX → RESULT

트러블슈팅 (문제해결과정)

STEP 01

문제

STEP 02

원인

STEP 03

해결

STEP 04

결과

트러블 슈팅 · CASE 01

경로 UI 미노출

PROBLEM

안내카드 미노출

길찾기를 시작했는데 하단에 있어야 할 안내카드가 나타나지 않았습니다. 사용자는 경로가 계산됐는지, 안내가 시작됐는지조차 알 수 없는 상태.

CAUSE

하단 패널 높이 제한

기존 하단 패널은 “ 내 제보 위치” 리스트가 이미 자리를 차지하고 있었고, 안내카드가 들어갈 영역이 **가려지는 구조**였습니다.

FIX

역할 재배치

- 하단 패널 = **길찾기 전용**으로 변경
- “ 내 제보” 리스트는 마이페이지 전용으로 이전
- 안내카드가 항상 하단에 표시되도록 확보



· 수정 후 이미지

트러블슈팅 · CASE 02

사고다발구역 API 호출 폭주

지도 이동 이벤트마다 KOROAD API가 반복 호출되던 문제

PROBLEM

지도 이동마다 API 반복 호출

사용자가 지도를 살짝만 움직여도 사고다발구역 API가 매번 호출되어, API 사용량과 렌더링 부하가 급격히 증가.

CAUSE

뷰포트 이벤트마다 조건 없이 재호출

뷰포트 변경 이벤트가 발생할 때마다 **별도 조건 없이** 사고다발 API를 다시 부르는 구조였습니다. 지도 이동은 초당 여러 번 발생하기에 부하가 누적.

FIX

2초 무동작 + 구 변경 시에만

- 2초간 지도 조작 없음 확인
- 구(區)가 바뀐 경우에만 호출
- 일정 줌 이상 축소 시에는 호출·표시 모두 중단

debounce(2s) + 구 변경 감지
+ zoom threshold

트러블슈팅 · CASE 03

사고 다발 구역 API 4종 분리

PROBLEM

전체 실패로 이어짐

KOROAD API 4종(보행자·자전거·이륜차·어린이보호구역) 중 **하나만 실패**해도 나머지 3종까지 함께 화면에 표시되지 않았습니다.

CAUSE

단일 실패 단위

4개 API 요청이 하나의 **단일 성공/실패 단위**로 묶여 있었습니다. 어느 하나가 reject되면 전체가 reject되는 구조.

```
await Promise.all([...])
→ 하나만 fail 해도 전체 catch
```

FIX

요청 단위 분리

- Promise 기반 **요청 단위 분리**
- 실패한 유형만 제외
- 나머지 유형은 정상 표시

```
Promise.allSettled([...])
→ 성공한 결과만 병합
```


트러블슈팅 · CASE 04

줌 아웃 시 격자 대량 생성

축소 레벨 제한 없이 모든 격자를 그리면서 렌더링 부담이 폭증한 문제

PROBLEM

과도한 축소 시 격자 폭주

전국 단위까지 축소하면 화면에 들어오는 격자가 수십만 단위로 늘어나 브라우저가 눈에 띄게 느려지고 프레임이 떨어졌습니다.

CAUSE

축소 레벨 상한 부재

최소 줌 레벨에 대한 제한 없이 뷰포트 안 **모든 격자**를 그리도록 되어 있어, 사용자가 축소만 하면 격자 수가 기하급수적으로 증가.

FIX

최소 줌 상한 + 렌더 제한

- **MAX_ZOOM_OUT = 9**
- 렌더링 격자 수 상한 적용
- 그 이상 축소 시 격자 표시 중단

```
if (zoom > MAX_ZOOM_OUT)
  skip grid render
```

트러블슈팅 • CASE 05

실시간 구독 중복 • 메시지 중복 표시

발생 상황

채팅방을 나갔다 들어올 때마다, 똑같은 안내 메시지가 메아리처럼 두 번, 세 번씩 중복해서 울림

오류 원인

채팅방 컴포넌트가 언마운트(화면 전환)될 때 기존 실시간 채널 구독을 해제하지 않아, 페이지를 이동할 때마다 구독 채널이 중첩되면서 전송된 메시지가 여러 번 렌더링되었음

해결

채팅방을 나갈 때는 기존 안테나를 확실하게 뽑아버리는 `Clean-up(cleanup 함수와 removeChannel)` 코드를 추가함

결과

오직 하나의 채널만 깔끔하게 유지되어 메시지가 정확히 한 번씩만 안전하게 수신되도록 함

채팅

홍길동

로그인 변경

관

관리자와의 대화

만든 사람: 14

관

관리자와의 대화

만든 사람: 3

관

관리자와의 대화

만든 사람: 13

관

관리자와의 대화

만든 사람: 11

공

공지사항

새 채팅방

+

트러블슈팅 • CASE 07

실시간 메시지의 닉네임 누락

발생 상황

실시간 채팅이 올라오는데 이름이 안 뜨고
undefined로만 뜨는 오류 발생

undefined: "안녕하세요"

오류 수정

메시지가 도착하는 순간, 시스템이 백그라운드에서 장부(user 테이블)를 빠르게 대조하는 **비동기 닉네임 매핑**(`resolveNickname`)을 구현함

오류 원인

Supabase Realtime 에서 `sender_id`와 메시지만
초고속으로 전송시켜주다 보니 그 번호를 가진
사람이 구체적으로 누구인지에 대해서 누락되었기
때문에 발생함

결과

즉시 이름을 매칭시켜 화면에
나오게 완성함

트러블슈팅 · CASE 09

경로 안내 UX 이슈

01 PROBLEM

안내 종료 후 패널 재노출

안내가 끝난 뒤에도 경로 선택 패널이 다시 표시.

FIX · 종료 시 패널 재노출 방지 상태 초기화

02 PROBLEM

롱프레스 주소 경로 검색 실패

일부 지역에서 롱프레스 지정 지점으로 경로 검색이 실패.

FIX · 주소 매칭 로직 수정

03 PROBLEM

안내 중 목적지 재지정 오류

새 목적지를 지정해도 기존 안내가 그대로 유지되는 문제.

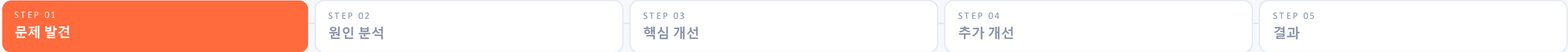
FIX · 기존 안내 세션을 정상 종료 후 새로 시작

04 PROBLEM

격자정보 오버플로우

일부 기종에서 격자정보 카드가 약 2px 화면 밖으로 넘어감.

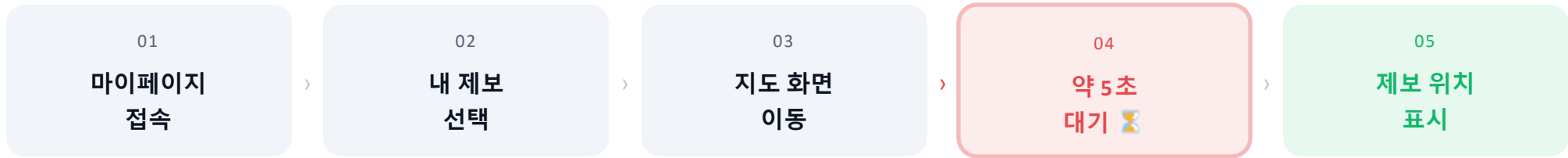
FIX · 격자정보를 정보 패널에 통합



트러블슈팅 · CASE 10

마이페이지에서 내 제보 위치 확인 시 상당시간 소요

USER FLOW · 문제 발생 과정



PROBLEM

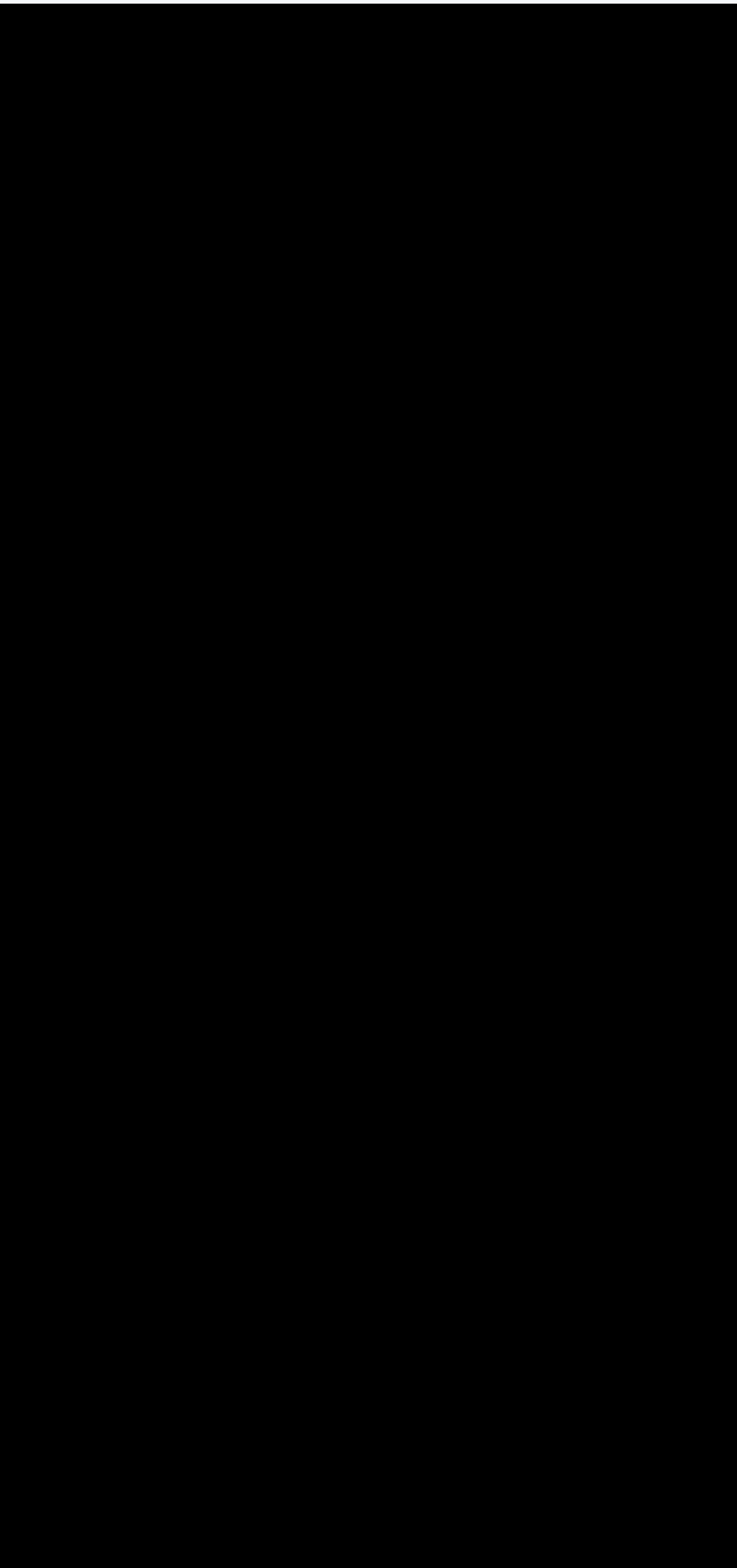
4단계에서 사용자가 “ 지도가 왜 안 움직이지?” 를 5초간 경험

기능은 정상 동작하지만, 응답이 즉시 오지 않아 체감 성능이 크게 떨어짐

MEASURED · BEFORE

제보 위치 확인까지

5.44초



STEP 01 문제 발견	STEP 02 원인 분석	STEP 03 핵심 개선	STEP 04 추가 개선	STEP 05 결과
------------------	------------------	------------------	------------------	---------------

트러블슈팅 · CASE 10

기존 방식의 문제점

ROOT CAUSE

pendingFocus 소비 지연

pendingFocus = 마이페이지에서 선택한 제보 위치를 지도 화면으로 이동한 후 Focus 하기 위해 저장하는 정보

마이페이지 → 지도 이동



pendingFocus 존재 → 현재 화면이 지도인지 확인

↓ 지도 활성화를 바로 인식하지 못함

재시도 · 대기 → 최대 5초 Timeout까지 대기



마침내 Focus 실행

TIME DISTRIBUTION

시간이 어디에 쌓였나

포커스 대기

focus wait

5.00초

지도 데이터 조회

map data

0.42초

카메라 선택

camera select

0.01초

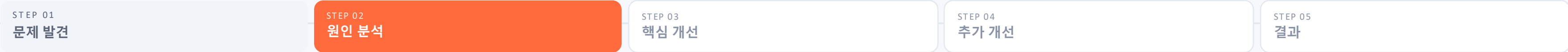
전체

total

5.44초

INSIGHT

실제 지도 처리(0.43초)보다 불필요한 포커스 대기(5.00초)가 전체 시간의 92%를 차지했다.



트러블슈팅 · CASE 10

기존 방식의 문제점

ROOT CAUSE

불필요한 데이터 조회 반복

- 사용자가 25m 거리를 이동하거나 60초마다 서버에 지도에 표시할 데이터를 서버에 요청
- 25만건 이상의 인프라 데이터를 매번 요청

GPS 현재 위치 탐색 및 이동에 동기적 처리

- 첫 지도 로딩 시 현재 위치를 탐색하고 카메라를 이동시키는데 동기적 처리
- 유저는 해당 행동이 끝날때까지 대기해야함

STEP 01
문제 발견

STEP 02
원인 분석

STEP 03
핵심 개선

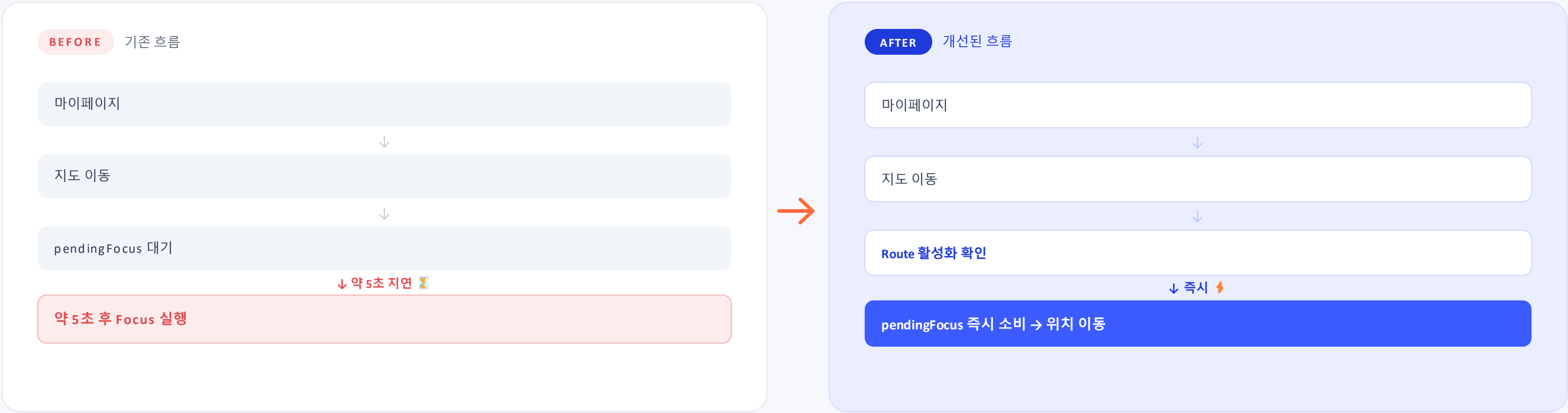
STEP 04
추가 개선

STEP 05
결과

트러블슈팅 · CASE 10

지도 활성화 직후, pendingFocus를 즉시 소비합니다

Route 활성화를 명시적으로 확인하고, 대기 없이 바로 포커스를 실행하도록 흐름을 재설계.



CORE RESULT

불필요한 포커스 대기를 제거

포커스 대기 · BEFORE → AFTER

5.00초 → 0.08초



트러블슈팅 · CASE 10

지도 초기 표시와 체감 대기시간을 위한 추가 개선

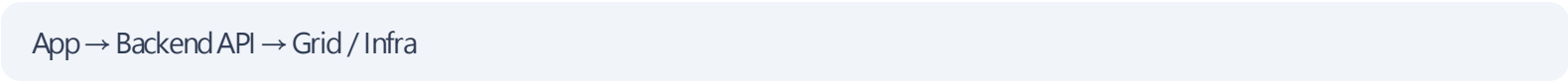
※ 아래 두 가지는 5.44초 → 0.11초 개선의 직접 원인은 아닙니다 —초기 표시·체감 대기시간을 위한 추가 구조 개선입니다.

STRUCTURE 01 SQLite

정적 데이터 로컬화

Grid·Infra 등 변경 빈도가 낮은 데이터를 앱 내부에 저장

BEFORE



AFTER



SERVER CALLS

↓ 감소

RANGE

5km → 30km

- 앱 최초 실행 시 데이터 버전 확인
- 변경 시에만 서버에서 갱신·반복 서버 통신 감소
- FCM 활용 - 신규 제보가 등록될 때 해당 정보를 수신하여 60초 마다 재탐색하는 구조 삭제

STRUCTURE 02 GPS Non-Blocking

GPS 비블로킹 처리

지도 표시가 GPS 응답을 기다리지 않도록 흐름 재설계

BEFORE



AFTER



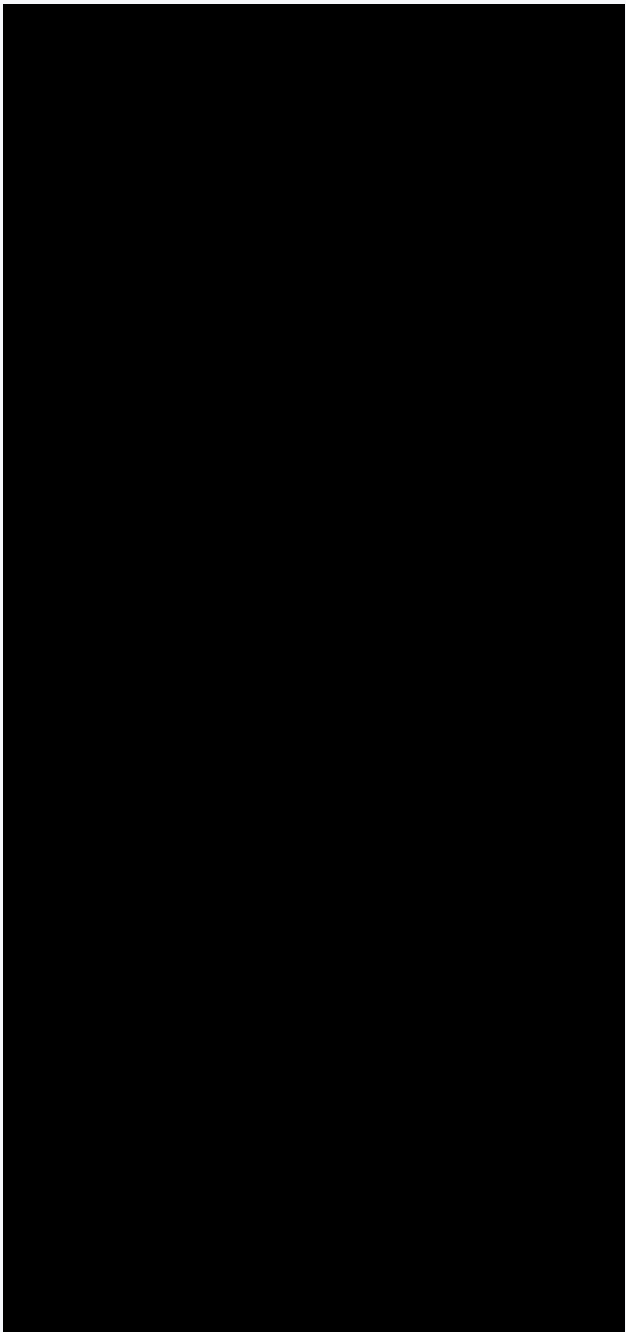
- 앱 종료·페이지 이동 시 **마지막 위치 저장**
- 마지막 위치로 지도를 먼저 표시
- GPS 확인·보정은 백그라운드 진행
- 사용자 체감 초기 대기시간 감소

트러블슈팅 · CASE 10

개선 이후



항목	Before	After	감소율
포커스 대기	5.00 초	0.08 초	98.5%
지도 데이터 조회	0.42 초	0.03 초	92.6%
제보 위치 이동	5.44 초	0.11 초	97.9%



CLOSING

불필요한 포커스 대기를 제거하고, 지도 데이터를 효율적으로 처리하는 구조로 개선하여 사용자가 직접 체감하는 지도 이동 성능을 개선했습니다.

07

RETROSPECTIVE

느낌점

마무리하며 — 팀원별 느낀점

MEMBER 01
01 윤영빈

해당 프로젝트를 통해 AI를 활용하여 진행하는 속도가 비약적으로 빨라진 것을 느꼈지만, 그만큼 자신의 구상을 구체화해야 하는 능력이 더욱 중요해졌다고 생각했습니다.

MEMBER 02
02 남협

비전공자의 한계를 딛고 추가 기초 교육과 실시간 채팅 구현을 거쳐 SAFETY 프로젝트 팀원으로 합류하여 직접 버그를 해결하고 주도적으로 기능을 개발하며 자립적인 학습 루틴을 체득했고, 이번 프로젝트를 통해 현업에서 나만의 멋진 서비스를 만드는 훌륭한 개발자가 되겠다는 강렬한 열망과 확신을 얻었습니다.

MEMBER 03
03 최정봉

AI를 활용하면서 생산능력이 많이 빨라졌음을 느꼈지만 이후 유지보수와 확장성을 위한 설계의 중요성을 더 실감했습니다.

MEMBER 04
04 박상우

AI 수업과 프로젝트를 통해 AI의 저력을 느꼈습니다. 처음 접하는 분야였지만 따라가기위해 노력을 하다보니 흥미를 느꼈고 잘하고 싶은 마음이 생겼습니다. 전공자인 동료들과 강사님의 도움으로 기초를 닦았고 실력을 향상 시켰습니다. 저의 첫 프로젝트를 좋은 동료들과 재밌게 하게되어 영광이었습니다!

MEMBER 05
05 최민기

[한줄 회고 입력란]

PUBLIC SAFETY MAP

감사합니다

3조 SAFETY

윤영빈 · 남협 · 최정봉 · 박상우 · 최민기